

# Tema 5: Tratamiento de datos

## Transacciones

|                                                            |                                                                                                       |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                       |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                         |

## Contenido

|                                                      |   |
|------------------------------------------------------|---|
| 1. Introducción .....                                | 2 |
| 2. Acceso concurrente a datos .....                  | 2 |
| 2.1. Niveles de aislamiento .....                    | 3 |
| 3. Gestión transacciones .....                       | 4 |
| 4. Ejemplos de nivel aislamiento/transacciones ..... | 4 |
| 4.1. Dirty read .....                                | 5 |
| 4.2. Nonrepeatable Read .....                        | 6 |
| 4.3. Phantom read.....                               | 7 |
| 5. Actividades .....                                 | 8 |

## 1. Introducción

Una transacción es un conjunto de sentencias SQL que se tratan como una sola instrucción (atómica), en otras palabras, o se hacen todas las operaciones o no se hace ninguna. Si todas las operaciones individuales se ejecutaron correctamente la transacción se puede confirmar (commit), o si hubo algún problema se puede abortar la transacción (rollback).

Trabajar con transacciones puede ser esencial para **mantener la integridad de los datos**. Por ejemplo, se puede dar el caso de que se descuente el stock de un producto antes de proceder a su envío, pero cuando se va a generar el pedido no hay conexión con la empresa de mensajería, por lo tanto es necesario "volver a reponer el stock", es decir, deshacer los cambios hechos previamente en la base de datos.

## 2. Acceso concurrente a datos

|                                                            |                                                                                                       |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                       |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                         |

Cuando dos transacciones acceden al mismo dato a la vez (conurrencia) pueden ocurrir alguno de los siguientes problemas:

- Dirty read (lectura sucia): una transacción lee datos modificados por otra transacción que todavía no ha hecho commit.
- Nonrepeatable Read (Lectura No Repetible): Ocurre cuando una transacción lee el mismo registro dos veces, pero obtiene valores diferentes debido a que otra transacción lo modificó entre las dos lecturas.
- Phantom Read (Lectura Fantasma). Ocurre cuando una transacción lee un conjunto de registros que cumplen con una condición, y luego otra transacción inserta o elimina registros que cumplen con la misma condición, lo que hace que la primera transacción vea "fantasmas" (registros nuevos o desaparecidos) en una segunda lectura.

## 2.1. Niveles de aislamiento

Cuando se trabaja con transacciones se pueden bloquear un conjunto de datos para evitar estos problemas. Según el nivel de concurrencia que se desee se pueden establecer cuatro niveles de aislamiento, es decir, definir como los cambios hechos por una transacción son visibles a otras transacciones.

- Read Uncommitted (lectura no acometida): permite lecturas sucias, lecturas no repetibles y lecturas fantasma.
- Read Committed (lectura acometida): permite lecturas no repetibles y lecturas fantasma, pero no lecturas sucias.
- Repeatable Read (lectura repetible): permite lecturas fantasmas, pero no lecturas sucias ni lecturas no repetibles.
- Serializable: No permite ninguna de las tres anomalías.

**Por defecto MySQL utiliza el nivel Repeatable Read.**

Se puede ajustar el nivel de aislamiento, a parte de con el fichero de configuración `my.cnf`, con la siguiente instrucción:

```
SET TRANSACTION ISOLATION LEVEL {READ UNCOMMITTED | READ COMMITTED
| REPEATABLE READ | SERIALIZABLE}
```

|                                                            |                                                                                                       |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                       |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                         |

### 3. Gestión transacciones

Normalmente los SGBD tienen activado el modo de confirmación por cada sentencia (AUTOCOMMIT=ON), para trabajar con transacciones con múltiples sentencias es necesario desactivar este modo automático. En MySQL se puede hacer de tres formas:

```
SET AUTOCOMMIT=0;

START TRANSACTION;

BEGIN WORK;
```

Para terminar la transacción es necesario aceptar (COMMIT) o rechazar los cambios (ROLLBACK).

Si la sesión se cierra abruptamente (por ejemplo el cliente pierde la conexión) MySQL realizará un rollback.

### 4. Ejemplos de nivel aislamiento/transacciones

A continuación, se muestra un ejemplo de cada nivel de aislamiento, crear la siguiente tabla y datos:

```
create table productos (
id int primary key,
nombre varchar(255),
stock int);

insert into productos (id, nombre, stock) values
```

|                                                            |                                                                                                        |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                        |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                          |

```
(1, 'Laptop', 10),
(2, 'Mouse', 20),
(3, 'Teclado', 30);
```

## 4.1. Dirty read

Ocurre cuando una transacción lee datos no confirmados de otra transacción.

1. Abre dos sesiones de MySQL.
2. En la sesión 1, inicia una transacción y actualiza un registro

```
-- Sesión 1
START TRANSACTION;
UPDATE productos SET stock = 5 WHERE id = 1;
```

3. En la sesión 2, antes de que la sesión 1 haga COMMIT, lee el mismo registro:

```
-- Sesión 2
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
SELECT stock FROM productos WHERE id = 1;
```

4. La sesión 2 verá el valor actualizado (5), aunque la sesión 1 aún no haya confirmado el cambio.
5. En la sesión 1, haz ROLLBACK
6. Si vuelves a leer el registro en la sesión 2, verás el valor original (10). La sesión 2 leyó un valor "sucio" que nunca debió haber existido.

|                                                            |                                                                                                        |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                        |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                          |

## 4.2. Nonrepeatable Read

Ocurre cuando una transacción lee el mismo registro dos veces, pero obtiene valores diferentes debido a que otra transacción lo modificó entre las dos lecturas.

1. Abre dos sesiones de MySQL.
2. En la sesión 1, inicia una transacción y lee un registro:

```
-- Sesión 1
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT stock FROM productos WHERE id = 2;
```

3. En la sesión 2, actualiza el mismo registro y haz COMMIT:

```
-- Sesión 2
START TRANSACTION;
UPDATE productos SET stock = 25 WHERE id = 2;
COMMIT;
```

4. En la sesión 1, vuelve a leer el mismo registro:

```
-- Sesión 1
SELECT stock FROM productos WHERE id = 2;
```

5. La sesión 1 verá un valor diferente en la segunda lectura, lo que demuestra una lectura no repetible.

|                                                            |                                                                                                        |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                        |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                          |

### 4.3. Phantom read

Ocurre cuando una transacción lee un conjunto de registros que cumplen con una condición, y luego otra transacción inserta o elimina registros que cumplen con la misma condición, lo que hace que la primera transacción vea "fantasmas" (registros nuevos o desaparecidos) en una segunda lectura.

1. Abre dos sesiones de MySQL.
2. En la sesión 1, inicia una transacción y lee un conjunto de registros

```
-- Sesión 1
START TRANSACTION;
SELECT * FROM productos WHERE stock > 15;
```

3. En la sesión 2, inserta un nuevo registro que cumple con la misma condición y haz COMMIT:

```
-- Sesión 2
START TRANSACTION;
INSERT INTO productos (id, nombre, stock) VALUES (4, 'Monitor', 40);
COMMIT;
```

4. En la sesión 1, vuelve a leer el mismo conjunto de registros:

```
-- Sesión 1
UPDATE productos SET stock = stock + 1;
SELECT * FROM productos WHERE stock > 15;
```

|                                                            |                                                                                                        |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                        |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                          |

5. La sesión 1 verá el nuevo registro insertado por la sesión 2, lo que demuestra una lectura fantasma.

## 5. Actividades

---

### *Actividad 1*

---

Dadas las siguientes tablas y datos:

```
create table vuelos (
    id_vuelo int primary key,
    destino varchar(50),
    asientos_libres int
);

create table reservas (
    id_reserva int auto_increment primary key,
    id_vuelo int,
    nombre_pasajero varchar(100),
    foreign key (id_vuelo) references vuelos(id_vuelo)
);

insert into vuelos values (101, 'Tokio', 1);
```

|                                                            |                                                                                                        |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| IES Isidra de Guzmán                                       |  IES Isidra de Guzmán |
| Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma | Curso 2025/2026                                                                                        |
| Tema 5: Tratamiento de datos                               | Transacciones                                                                                          |

- El pasajero Jose Luis quiere reservar en el vuelo a Tokio, realizar el script sql para gestionar esa reserva indicando que nivel de aislamiento es necesario.
- Posteriormente el pasajero Jose Luis no puede viajar a Tokio por lo pide anular su reserva, realizar el script sql para gestionar este cambio.